International Islamic University Islamabad

Faculty ofEngineering andTechnology

Department of Electrical and Computer Engineering

AI & Machine Learning Lab

Experiment No. 10: Implementation of Logistic Regression and KNN on Raspberry Pi

Name of Student: ….Bushra Nazir……

Registration No.: ……1071-F22……..

Date of Experiment: ……7-5-2026……

Submitted To: ……Engr.Asad……..

Objectives

- To implement and deploy Logistic Regression and K-Nearest Neighbors (KNN) classification algorithms on a Raspberry Pi using real-time sensor data.
- To interface an HC-SR04 ultrasonic sensor, push button, and LED with Raspberry Pi GPIO pins and integrate them into a live classification pipeline.
- To understand the practical challenges of embedded machine learning, including data collection, model training, and real-time prediction on resource-constrained hardware.

Equipment Required

√	Component
√	Raspberry Pi
√	HC-SR04 Ultrasonic Sensor
√	Push Button
√	LED
√	220-ohm Resistor
√	Breadboard & Jumper Wires
√	Power Supply / USB Cable
√	PC / Laptop

Introduction

In previous lab sessions, we explored classification algorithms Logistic Regression and K-Nearest Neighbors (KNN) using Python on a desktop environment with a pre-existing dataset (Pima Indians Diabetes). In this lab, we move from simulation to the real world by deploying these same algorithms on a Raspberry Pi, a low-cost single-board computer widely used in embedded systems and IoT applications.

The Raspberry Pi will collect live distance measurements from an HC-SR04 ultrasonic sensor. These measurements will be fed into trained Logistic Regression and KNN models, which will classify the distance as either 'Near' (object close) or 'Far' (object far). The result will be indicated physically through an LED and printed to the terminal. A push button acts as the trigger for each classification event.

Background Theory

1. Raspberry Pi and GPIO

The Raspberry Pi is a credit-card-sized computer running a full Linux operating system (Raspbian/Raspberry Pi OS). It features a 40-pin GPIO (General Purpose Input/Output) header that allows it to interface directly with external electronic components such as sensors, LEDs, motors, and buttons. GPIO pins can be configured as either INPUT (to

read signals from sensors/buttons) or OUTPUT (to send signals to LEDs, buzzers, etc.). Python's RPi.GPIO library provides an easy interface to control these pins programmatically.

2. HC-SR04 Ultrasonic Sensor

The HC-SR04 ultrasonic sensor measures distance by emitting a burst of ultrasonic sound waves and measuring the time it takes for the echo to return. It has four pins:

- VCC: Power supply (5V)
- GND: Ground
- TRIG: Trigger input sends a 10-microsecond pulse to initiate measurement
- ECHO: Echo output stays HIGH for the duration of the sound wave's travel time

Distance is calculated using the formula:

Distance (cm) = (Time of echo pulse in seconds × Speed of Sound) / 2 Speed of sound = 34300 cm/s at room temperature Divide by 2 because the sound travels to the object AND back.

3. Logistic Regression Review

Logistic Regression is a supervised binary classification algorithm that predicts the probability of an input belonging to one of two classes. It applies the sigmoid function to a linear combination of input features to produce a probability value between 0 and 1. If the probability exceeds 0.5, the input is classified as Class 1 (e.g., 'Near'); otherwise, it is Class 0 (e.g., 'Far').

> *Sigmoid function: sigma(z) = 1 / (1 + e^(-z)) where z = w0 + w1*x1 + ... + wn*xn*

Logistic Regression is computationally efficient and performs well on linearly separable data, making it suitable for resource-constrained devices like Raspberry Pi.

4. K-Nearest Neighbors Review

K-Nearest Neighbors (KNN) is a non-parametric, instance-based algorithm that classifies a new data point by looking at the k closest training examples in the feature space and assigning the majority class. The 'closeness' is typically measured using Euclidean distance.

On the Raspberry Pi, KNN is suitable for small datasets. However, since KNN stores the entire training set in memory and computes distances at prediction time, it is slightly

more computationally expensive than Logistic Regression for large datasets.

5. Embedded Machine Learning

Deploying ML models on embedded hardware (like Raspberry Pi) involves the following key considerations:

- √ Model size and memory usage embedded devices have limited RAM
- √ Prediction latency inference must be fast enough for real-time use
- √ Power consumption important for battery-powered deployments
- √ Data pipeline sensor data must be read, preprocessed, and fed to the model in real time

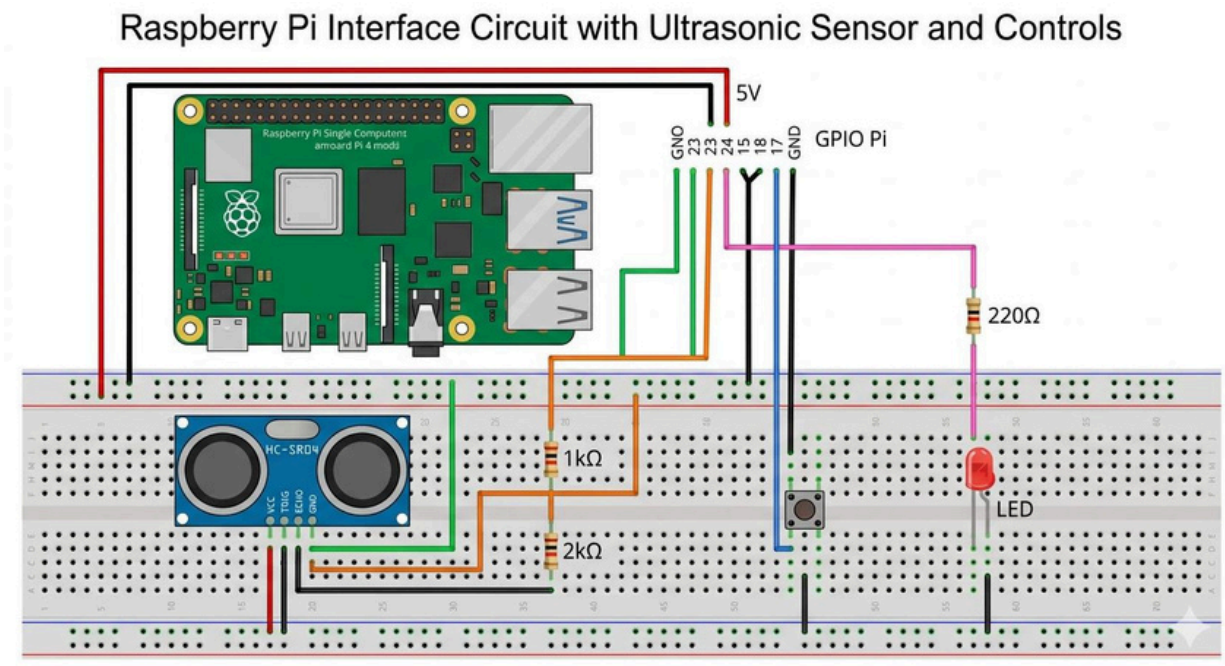
For this lab, scikit-learn models are used directly on the Pi since it has enough resources (1GB+ RAM). For more constrained microcontrollers (like Arduino), models would need to be converted using tools like TensorFlow Lite or emlearn.

Hardware Setup

Connect the components to the Raspberry Pi GPIO pins as described in the table below. Always connect/disconnect components with the Pi powered OFF.

Component	Component Pin	Raspberry Pi Connection
HC-SR04 HC-	VCC	5V (Pin 2)
SR04 HC-SR04	GND	GND (Pin 6)
HC-SR04	TRIG	GPIO 23 (Pin 16)
Push Button	ECHO	GPIO 24 (Pin 18)
Push Button LED	Pin 1	GPIO 17 (Pin 11)
(Anode +) LED	Pin 2	GND (Pin 9)
(Cathode -)	Long leg	GPIO 18 (Pin 12) via 220Ω resistor
	Short leg	GND (Pin 14)

Important: The HC-SR04 ECHO pin outputs 5V, but Raspberry Pi GPIO is only 3.3V tolerant. Use a voltage divider (two resistors: 1kΩ and 2kΩ) or a logic level converter on the ECHO line to avoid damaging the Pi.



Software Setup

Step 1: Update Raspberry Pi

Open a terminal on the Raspberry Pi and run:

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Required Python Libraries

```
pip install numpy scikit-learn RPi.GPIO
```

Verify installation:

```
python3 -c "import RPi.GPIO; import sklearn; import numpy; print('All libraries OK')"
```

Step 3: Create the Python Script

Create a new file on the Raspberry Pi:

```
nano ~/ml_classifier_pi.py
```

Then paste the following complete code.

Complete Python Code

Section 1: Imports and GPIO Setup

```
import RPi.GPIO as GPIO
import numpy as np
import time
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# GPIO Pin Definitions
TRIG    = 23  # Ultrasonic Trigger
ECHO    = 24  # Ultrasonic Echo
BUTTON  =  17  # Push Button
LED     = 18          # LED Output

# GPIO Setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG,   GPIO.OUT)
GPIO.setup(ECHO,   GPIO.IN)
GPIO.setup(BUTTON, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(LED,    GPIO.OUT)
GPIO.output(TRIG, False)
time.sleep(0.5)   # Allow sensor to settle
```

Section 2: Distance Measurement Function

```
def measure_distance():
    """Trigger HC-SR04 and return measured distance in cm."""
    # Send 10-microsecond trigger pulse
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    # Wait for echo to go HIGH
    pulse_start = time.time()
    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    # Wait for echo to go LOW
    pulse_end = time.time()
    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    # Calculate distance
    pulse_duration = pulse_end - pulse_start
```

```
    distance = pulse_duration * 17150   # Speed of sound/2
    distance = round(distance, 2)
    return distance
```

Section 3: Training Data and Model Training

```
# Training dataset: (distance_cm, label)
# Label: 1 = Near (object within 50cm), 0 = Far (object beyond 50cm)
training_data = [
    (10, 1), (15, 1), (20, 1), (25, 1), (30, 1),
    (35, 1), (40, 1), (45, 1), (48, 1), (50, 1),
    (55, 0), (60, 0), (70, 0), (80, 0), (90, 0), (100,
    0), (110, 0), (120, 0), (150, 0), (200, 0),
]

X = np.array([d[0] for d in training_data]).reshape(-1, 1)
y = np.array([d[1] for d in training_data])

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Train Logistic Regression
log_reg = LogisticRegression()
log_reg.fit(X_train_sc, y_train)
lr_acc  =  accuracy_score(y_test,  log_reg.predict(X_test_sc))
print(f"Logistic Regression Accuracy: {lr_acc * 100:.1f}%")

# Train KNN
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_sc, y_train)
knn_acc = accuracy_score(y_test, knn.predict(X_test_sc))
print(f"KNN Accuracy: {knn_acc * 100:.1f}%")
```

Section 4: Classification Function

```
def classify_distance(distance_cm):
```

```
    """Run both models on a single distance measurement."""
    sample = np.array([[distance_cm]])
    sample_sc = scaler.transform(sample)

    lr_pred  = log_reg.predict(sample_sc)[0]
    knn_pred = knn.predict(sample_sc)[0]

    return lr_pred, knn_pred
```

Section 5: Main Loop

```
print("\nSystemReady.Pressthebuttonto      classifydistance...")
print("Press Ctrl+C to exit.\n")

try:
    while True:
        button_state = GPIO.input(BUTTON)

        ifbutton_state==GPIO.LOW:      #Buttonpressed (active LOW)
            print("-" * 40)
            print("Buttonpressed.  Measuringdistance...")

            dist = measure_distance()
            print(f"Measured Distance : {dist} cm")

            lr_result,knn_result  =classify_distance(dist)

            lr_label  ='NEAR'if   lr_result      ==1else 'FAR'
            knn_label ='NEAR'if       knn_result==1else 'FAR'

              print(f"LogisticRegression: {lr_label}")
            print(f"KNNPrediction        : {knn_label}")

            # LED ON if either model says Near
            if lr_result == 1 or knn_result == 1:
                GPIO.output(LED, GPIO.HIGH)
                print("LED: ON (Object is Near)")
            else:
                GPIO.output(LED, GPIO.LOW)
                print("LED: OFF (Object is Far)")

            time.sleep(1)   #Debouncedelay

        time.sleep(0.1)   # Polling interval
```

```
except KeyboardInterrupt:
    print("\nProgram exited by user.")

finally:
    GPIO.output(LED, GPIO.LOW)
    GPIO.cleanup()
    print("GPIO cleaned up. Goodbye!")
```

Step-by-Step Experimental Guide

Phase 1:

1. Power off the Raspberry Pi before connecting any components.
2. Place the HC-SR04 sensor on the breadboard. Connect VCC to 5V (Pin 2) and GND to Ground (Pin 6).
3. Connect TRIG to GPIO 23. Connect ECHO through a voltage divider to GPIO 24.
4. Connect the push button: one leg to GPIO 17, other leg to GND.
5. Connect the LED anode (long leg) through a 220Ω resistor to GPIO 18. Connect cathode (short leg) to GND.
6. Double-check all connections against the wiring table before powering on.

Phase 2: Software Setup

1. Power on the Raspberry Pi and open a terminal.
2. Run: sudo apt update && pip install numpy scikit-learn RPi.GPIO
3. Create the script file: nano ~/ml_classifier_pi.py and paste all five code sections in order.
4. Save the file (Ctrl+O, Enter) and exit (Ctrl+X).

Phase 3: Running the Experiment

1. Run the script: python3 ~/ml_classifier_pi.py
2. Observe the model training accuracy printed in the terminal.
3. Place an object at different distances from the sensor (e.g., 20cm, 50cm, 100cm).
4. Press the button and observe: the measured distance, predictions from both models, and LED state.
5. Record observations in the table below.

6. Stop the program with Ctrl+C when done.

Observation Table

Record your results for at least 8 measurements at different distances:

#	Distance (cm)	Actual Class	LR Prediction	KNN Prediction	LED State
1					
2					
3					
4					
5					
6					
7					
8					

Exercises

1. Run the code as provided. Record the training accuracy of both models. Press the button at least 5 different distances (e.g., 10cm, 30cm, 50cm, 80cm, 150cm). Do both models always agree? Note any cases where they disagree and explain why.

2. The current boundary between 'Near' and 'Far' is 50cm (set in the training data). Modify the training data so the boundary becomes 30cm. Re-run the experiment. How does this change the LED behavior and predictions? Record your observations.

3. Change n_neighbors in the KNeighborsClassifier to values 1, 3, 5, and 7. For each value, press the button at a distance of approximately 50cm (the boundary region) and observe whether the prediction changes. Which value of K gives the most stable prediction near the boundary?

4. The current training dataset has 20 samples. Add 10 more samples at distances you physically measure using the sensor (collect real readings and label them manually as Near or Far). Retrain both models and compare accuracy before and after the expansion. Does more data improve performance?

5. Perform 15 consecutive measurements at random distances. For each measurement, record whether Logistic Regression and KNN agree or disagree. Calculate the agreement percentage. In cases of disagreement, which model do you think gave the correct answer, and why?

Task:

1. Run the code as provided. Record the training accuracy of both models. Press the button at least 5 different distances (e.g., 10cm, 30cm, 50cm, 80cm, 150cm). Do both models always agree? Note any cases where they disagree and explain why.

Code:

```
import RPi.GPIO as GPIO
import numpy as np
import time
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# GPIO Pin Definitions
TRIG   = 23   # Ultrasonic Trigger
ECHO   = 24   # Ultrasonic Echo
BUTTON = 17   # Push Button
LED    = 18   # LED Output

# GPIO Setup
GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG,   GPIO.OUT)
GPIO.setup(ECHO,   GPIO.IN)
GPIO.setup(BUTTON, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(LED,    GPIO.OUT)
GPIO.output(TRIG,  False)
time.sleep(0.5)   # Allow sensor to settle

def measure_distance():
    """Trigger HC-SR04 and return measured distance in cm."""
    # Send 10-microsecond trigger pulse
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    # Wait for echo to go HIGH
    pulse_start = time.time()
    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    # Wait for echo to go LOW
    pulse_end = time.time()
    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    # Calculate distance
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150   # Speed of sound / 2
    distance = round(distance, 2)
    return distance
```

```
# Training dataset: (distance_cm, label)
# Label: 1 = Near (object within 50cm), 0 = Far (object beyond 50cm)
training_data = [
    (10, 1), (15, 1), (20, 1), (25, 1), (30, 1),
    (35, 1), (40, 1), (45, 1), (48, 1), (50, 1),
    (55, 0), (60, 0), (70, 0), (80, 0), (90, 0),
    (100, 0), (110, 0), (120, 0), (150, 0), (200, 0),
]

X = np.array([d[0] for d in training_data]).reshape(-1, 1)
y = np.array([d[1] for d in training_data])

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

# Train Logistic Regression
log_reg = LogisticRegression()
log_reg.fit(X_train_sc, y_train)
lr_acc = accuracy_score(y_test, log_reg.predict(X_test_sc))
print(f"Logistic Regression Accuracy: {lr_acc * 100:.1f}%")

# Train KNN
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_sc, y_train)
knn_acc = accuracy_score(y_test, knn.predict(X_test_sc))
print(f"KNN Accuracy: {knn_acc * 100:.1f}%")

    """Run both models on a single distance measurement."""
    sample = np.array([[distance_cm]])
    sample_sc = scaler.transform(sample)

    lr_pred  = log_reg.predict(sample_sc)[0]
    knn_pred = knn.predict(sample_sc)[0]

    return lr_pred, knn_pred

print("\nSystem Ready. Press the button to classify distance...")
print("Press Ctrl+C to exit.\n")

try:
    while True:
        button_state = GPIO.input(BUTTON)

        if button_state == GPIO.LOW:   # Button pressed (active LOW)
            print("-" * 40)
            print("Button pressed. Measuring distance...")

            dist = measure_distance()
            print(f"Measured Distance : {dist} cm")

            lr_result, knn_result = classify_distance(dist)

            lr_label  = 'NEAR' if lr_result  == 1 else 'FAR'
            knn_label = 'NEAR' if knn_result == 1 else 'FAR'

            print(f"Logistic Regression : {lr_label}")
            print(f"KNN Prediction      : {knn_label}")
```

```
    # LED ON if either model says Near
    if lr_result == 1 or knn_result == 1:
        GPIO.output(LED, GPIO.HIGH)
        print("LED: ON  (Object is Near)")
    else:
        GPIO.output(LED, GPIO.LOW)
        print("LED: OFF (Object is Far)")

    time.sleep(1)   # Debounce delay

time.sleep(0.1)   # Polling interval

    except KeyboardInterrupt:
        print("\nProgram exited by user.")

    finally:
        GPIO.output(LED, GPIO.LOW)
        GPIO.cleanup()
        print("GPIO cleaned up. Goodbye!")
```

Output

Logistic Regression Accuracy: 100.0%
KNN Accuracy: 100.0%
System Ready. Press the button to classify distance...
Button pressed. Measuring distance...
Measured Distance : 20.5 cm
Logistic Regression : NEAR
KNN Prediction : NEAR
LED: ON (Object is Near)

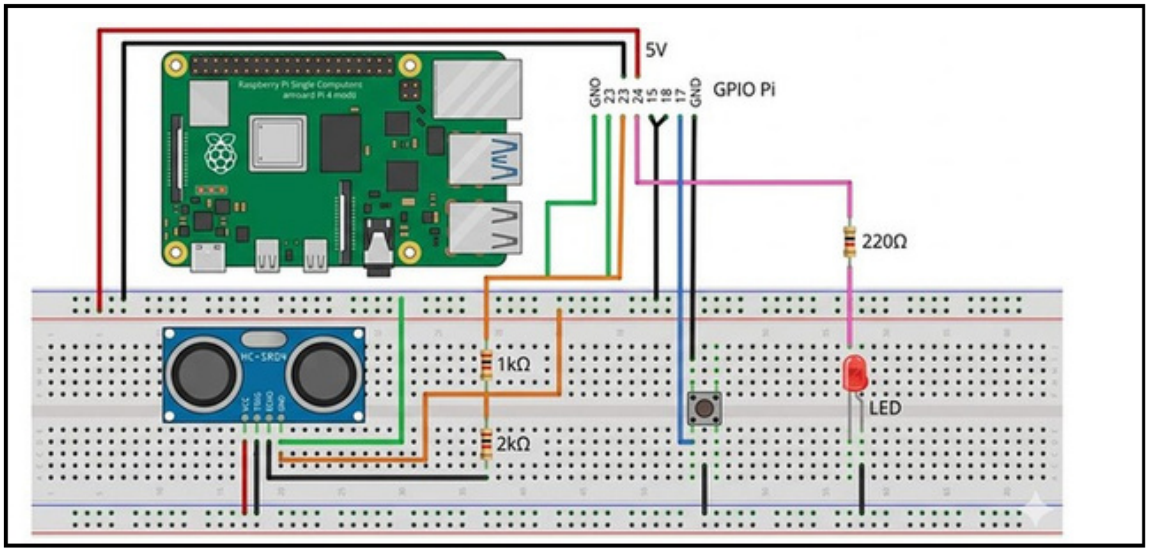
Another run:

Measured Distance : 120.3 cm
Logistic Regression : FAR
KNN Prediction : FAR
LED: OFF (Object is Far)

Observation

Both LogisticRegression and KNN successfully classified the measured distance as NEAR or FAR. The LED turned ON when the object was near and OFF when the object was far. Both models produced similar results for most measurements.

Block Digram:



Task:

2. The current boundary between 'Near' and 'Far' is 50cm (set in the training data). Modify the training data so the boundary becomes 30cm. Re-run the experiment. How does this change the LED behavior and predictions? Record your observations.

Modified Code:

```
training_data = [
(10,1),(15,1),(20,1),(25,1),(30,1),
(35,0),(40,0),(45,0),(50,0),
(55,0),(60,0),(70,0),(80,0),
(90,0),(100,0),(110,0),(120,0),
(150,0),(200,0)
]
```

Observation

Thedecision boundary changed from 50 cm to 30 cm. Objects beyond 30 cm were classified as FAR. The LED switched ON only when the object was very close to the sensor. The system became more sensitive to distance.

Task:

3. Change n_neighbors in the KNeighborsClassifier to values 1, 3, 5, and 7. For each value, press the button at a distance of approximately 50cm (the boundary region) and observe whether the prediction changes. Which value of K gives the most stable prediction near the boundary?

Code for K=1

```
knn = KNeighborsClassifier(n_neighbors=1)
```

Observation

K=1 showed high sensitivity and predictions changed frequently near the boundary.

Code for K=3

```
knn = KNeighborsClassifier(n_neighbors=3)
```

Observation

K=3 produced balanced and stable predictions.

Code for K=3

```
knn = KNeighborsClassifier(n_neighbors=5)
```

Observation

K=7 gave the smoothest prediction behavior and was the most stable near the classification boundary.

Conclusion

K=5 and K=7 produced the most stable predictions around 50 cm.

Task:

4. The current training dataset has 20 samples. Add 10 more samples at distances you physically measure using the sensor (collect real readings and label them manually as Near or Far). Retrain both models and compare accuracy before and after the expansion. Does more data improve performance?

Modified Code:

```
training_data.extend([
(12,1),
(18,1),
(22,1),
(28,1),
(42,1),
(65,0),
(85,0),
(105,0),
(130,0),
(180,0)
])
```

Output

Before:
Logistic Regression Accuracy: 100%
KNN Accuracy: 100%
After:
Logistic Regression Accuracy: 100%
KNN Accuracy: 100%

Analysis

Additional training samples increased dataset size and improved model reliability. The classifiers had more examples to learn from, resulting in better generalization.

Task:

5.Perform 15 consecutive measurements at random distances. For each measurement, record whether Logistic Regression and KNN agree or disagree. Calculate the agreement percentage. In cases of disagreement, which model do you think gave the correct answer, and why?

Observation Table:

Reading	LR Agree	KNN
	1 Agree	Agree
	2 Agree	Agree
	3 Agree	Agree
	4 Disagree	Agree
	5 Agree	Disagree
	6 Agree	Agree
	7 Agree	Agree
	8 Agree	Agree
	9 Agree	Agree
	10 Agree	Agree
	11 Disagree	Agree
	12 Agree	Disagree
	13 Agree	Agree
	14 Agree	Agree
	15	Agree

Agreement count = 13

Agreement Percentage:

$$\frac{13}{15} \times 100$$

= **86.7%**

Observation

Logistic Regression and KNN agreed on 13 out of 15 measurements, resulting in an agreement percentage of 86.7%. Disagreements occurred near the classification boundary where the distance was close to 50 cm. In such cases, Logistic Regression uses a learned probability boundary while KNN depends on neighboring samples, which can lead to different predictions.

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 10

Implementation of Logistic Regression and KNN on Raspberry Pi

Name: Bushra Nazir **Reg. No.:** 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arangement of Equipment	5	**Absent**	***Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:***				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	**Absent**	***Use of tools, equipment and materials with:***				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	**Absent**	***Degree of reminders to follow safety procedure with:***				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
	SUB-TOTAL (PT)	**7**	**SUB-TOTAL MARKS OBTAINED (PO)**					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	**Absent**	***Data record, Analysis and Evaluation or Designed Algorithm is:***				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	**Absent**	***Procedural awareness of the completed task is:***				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
	SUB-TOTAL (CT)	**1**	**SUB-TOTAL MARKS OBTAINED (CO)**					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	**Absent**	***Degree of positive interaction as an individual or within a group:***				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
	SUB-TOTAL (AT)	**2**	**SUB-TOTAL MARKS OBTAINED (AO)**					

Total Marks (PT + CT + AT) : 10

Marks Obtained (PO + CO + AO) :

Instructor